

Comparative Analysis of Classical and Graph Neural Network Approaches for Community Detection in Citation Networks

Emre Yıldız

Department of Computer Engineering

Ege University

İzmir, Turkey

emre.yildiz.dev@hotmail.com

Abstract—Community detection in complex networks is a fundamental problem in network analysis with applications ranging from social network analysis to biological systems. This paper presents a comprehensive comparative study of classical community detection algorithms and modern Graph Neural Network (GNN) approaches applied to the Cora citation network dataset. We implement and evaluate three classical algorithms (Louvain and Label Propagation) alongside three GNN architectures (Graph Convolutional Networks, GraphSAGE, and Graph Attention Networks) in both supervised and unsupervised settings. Our experimental framework integrates Neo4j graph database for efficient data management and classical algorithm execution, while PyTorch Geometric provides the foundation for GNN implementations. Results demonstrate that semi-supervised GNN approaches significantly outperform classical methods, achieving Adjusted Rand Index scores of 0.87, 0.86, and 0.86 for GAT, GraphSAGE, and GCN respectively, compared to 0.24 and 0.15 for Louvain and Label Propagation algorithms. The study provides insights into the effectiveness of different approaches for community detection in academic citation networks.

Index Terms—community detection, graph neural networks, citation networks, Neo4j, Louvain algorithm, label propagation

I. INTRODUCTION

Community detection is a crucial task in network analysis that aims to identify groups of nodes that are more densely connected to each other than to nodes in other groups. In the context of citation networks, communities often represent research areas or academic disciplines where papers within the same community share similar topics or methodologies.

The Cora dataset, consisting of 2,708 machine learning papers classified into seven categories (Neural Networks, Probabilistic Methods, Genetic Algorithms, Theory, Case-Based, Reinforcement Learning, and Rule Learning), provides an excellent benchmark for evaluating community detection algorithms. Each paper is represented by a 1,433-dimensional binary feature vector indicating the presence or absence of specific words, with citation relationships forming the network edges.

Traditional community detection algorithms such as the Louvain algorithm and Label Propagation have been widely used due to their computational efficiency and interpretability. However, recent advances in deep learning have introduced

Graph Neural Networks (GNNs) that can leverage both network structure and node features for improved community detection performance.

This work contributes a comprehensive comparison framework that evaluates both classical and modern approaches on the same dataset, providing insights into their relative strengths and limitations. Our implementation integrates multiple technologies: Neo4j for graph data management and classical algorithm execution, and PyTorch Geometric for GNN model development and training.

II. RELATED WORK

Community detection has been extensively studied in network science. Classical approaches include modularity optimization methods like the Louvain algorithm [1], which iteratively optimizes modularity to identify communities. Label Propagation [2] represents another class of algorithms that propagate node labels through the network until convergence.

Graph Neural Networks have emerged as powerful tools for node classification and community detection. Graph Convolutional Networks (GCNs) [3] extend convolutional operations to graphs, while GraphSAGE [4] introduces sampling and aggregation for scalable learning. Graph Attention Networks (GATs) [5] incorporate attention mechanisms to weight neighbor contributions differently.

Recent studies have shown that GNNs can effectively combine structural and feature information for community detection tasks, often outperforming traditional methods when labeled data is available.

III. METHODOLOGY

A. System Architecture

Our experimental framework consists of several integrated components designed to facilitate comprehensive comparison of community detection approaches:

Data Management Layer: Neo4j graph database serves as the central data repository, storing the Cora dataset's nodes (papers) and edges (citations). This choice enables efficient execution of graph algorithms and provides native support for community detection algorithms.

Classical Algorithm Layer: Implemented within Neo4j using the Graph Data Science library, including Louvain and Label Propagation algorithms. These algorithms operate directly on the graph structure stored in the database.

Deep Learning Layer: PyTorch Geometric framework handles GNN model implementation and training. Models include GCN, GraphSAGE, and GAT architectures, each tested in both supervised and unsupervised settings.

Evaluation and Visualization Layer: Comprehensive evaluation metrics and visualization tools for comparing algorithm performance and analyzing community structures.

B. Dataset Description

The Cora dataset consists of:

- 2,708 machine learning papers
- 5,429 citation relationships
- 1,433 binary features per paper
- 7 subject classes (ground truth communities)

Figure 1 shows the distribution of papers across different subject categories, revealing class imbalance with Neural Networks being the largest category (818 papers) and Rule Learning the smallest (180 papers).

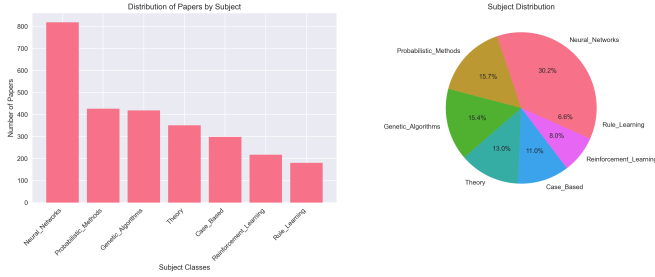


Fig. 1: Distribution of papers across subject categories in the Cora dataset

C. Classical Community Detection Algorithms

1) *Louvain Algorithm:* The Louvain algorithm optimizes modularity through a two-phase iterative process. In our implementation, the algorithm identified 102 communities with a final modularity score of 0.82, indicating strong community structure. The hierarchical nature of the algorithm allows for multi-resolution community detection.

2) *Label Propagation Algorithm:* Label Propagation assigns labels to nodes based on the majority label among their neighbors. Our implementation converged after 8 iterations, identifying 224 communities. The larger number of communities suggests a more fine-grained partitioning compared to Louvain.

D. Graph Neural Network Models

1) *Graph Convolutional Network (GCN):* GCN aggregates information from node neighborhoods through spectral convolution operations. Our implementation uses a two-layer architecture with 64 hidden units and ReLU activation functions.

2) *GraphSAGE:* GraphSAGE samples and aggregates features from node neighborhoods, enabling inductive learning on large graphs. We employ mean aggregation with the same architectural configuration as GCN.

3) *Graph Attention Network (GAT):* GAT incorporates attention mechanisms to weight neighbor contributions. Our implementation uses 4 attention heads in the first layer, allowing the model to focus on the most relevant neighbors for each node.

E. Training Strategies

Unsupervised Learning: Models learn node representations through contrastive loss, encouraging connected nodes to have similar embeddings. Community assignments are obtained through k-means clustering of learned embeddings.

Semi-supervised Learning: Models are trained on a subset of labeled nodes (20% training, 10% validation, 70% test) using cross-entropy loss for direct community classification.

IV. TECHNICAL IMPLEMENTATION

A. Neo4j Graph Database Integration

Neo4j serves as the backbone of our experimental framework, providing both data storage and native graph algorithm execution capabilities. The integration involves several key components:

Database Schema Design: The Cora dataset is modeled using a simple but effective schema where papers are represented as nodes with properties including unique identifiers, subject classifications, and feature vectors. Citation relationships are modeled as directed edges with the CITES relationship type.

Data Loading Pipeline: Raw Cora data is preprocessed and loaded into Neo4j using Cypher queries. The loading process involves two stages: first, creating paper nodes with their attributes, and second, establishing citation relationships between papers.

Graph Data Science (GDS) Library: Neo4j's GDS library provides optimized implementations of classical community detection algorithms. We leverage this for executing Louvain and Label Propagation algorithms directly within the database, eliminating the need for data export/import operations.

The following Cypher query demonstrates the Louvain algorithm execution:

```
1 CALL gds.louvain.write(
2   'cora-graph',
3   {
4     writeProperty: 'louvainCommunityId',
5     includeIntermediateCommunities: true
6   }
7 )
8 YIELD communityCount, modularities
```

Listing 1: Louvain Algorithm Execution in Neo4j

Data Export for GNN Processing: Graph data is extracted from Neo4j and converted to PyTorch Geometric format, maintaining consistency between classical and GNN approaches. This involves mapping Neo4j node IDs to tensor indices and preserving community assignments from classical algorithms for comparison.

B. Software Dependencies and Tools

Our implementation leverages a comprehensive software stack detailed in the project's requirements.txt file:

Core Deep Learning Framework:

- **PyTorch (2.0.0)**: Primary deep learning framework providing tensor operations and automatic differentiation
- **PyTorch Geometric (2.3.0)**: Specialized library for geometric deep learning on graphs, providing GNN layer implementations

Graph Database and Connectivity:

- **Neo4j (5.0.0)**: Graph database for data storage and classical algorithm execution
- **py2neo (2021.2.0)**: Python driver for Neo4j database connectivity

Data Processing and Analysis:

- **Pandas (1.5.0)**: Data manipulation and analysis, particularly for handling experimental results
- **NumPy (1.20.0)**: Numerical computing foundation for array operations
- **NetworkX (2.8.0)**: Graph analysis and manipulation for visualization preprocessing
- **Scikit-learn (1.2.0)**: Machine learning utilities including evaluation metrics and clustering algorithms

Visualization and Analysis:

- **Matplotlib (3.6.0)**: Static plotting for performance charts and analysis figures
- **Seaborn (0.12.0)**: Statistical visualization enhancement for matplotlib
- **Plotly (5.15.0)**: Interactive visualization for network graphs and dashboards

Development and Utility Libraries:

- **tqdm (4.64.0)**: Progress bars for long-running training processes
- **python-dotenv (1.0.0)**: Environment variable management for configuration
- **Requests (2.28.0)**: HTTP library for dataset downloading
- **Jupyter (1.0.0)**: Interactive development environment for analysis

C. Key Code Implementation

1) *Neo4j Database Manager*: The Neo4jManager class encapsulates all database operations, providing a clean interface between the graph database and Python analysis code:

```
1 class Neo4jManager:
2     def __init__(self):
3         self.driver = GraphDatabase.driver(
4             NEO4J_CONFIG["uri"],
5             auth=(NEO4J_CONFIG["username"],
6                 NEO4J_CONFIG["password"])
7         )
8
9     def load_cora_dataset(self):
10         query_create_papers = """
11         LOAD CSV WITH HEADERS FROM 'file:///cora.
12         content'
13         AS row FIELDTERMINATOR '\t'
14         CREATE (p:Paper {
```

```
14         id: toInteger(row.paper_id),
15         subject: row.subject,
16         features: split(row.words, ','))
17     })
18     """
19     # Execute queries for data loading
```

Listing 2: Neo4j Database Manager Implementation

2) *GNN Model Architecture*: The GNN models are implemented using PyTorch Geometric's modular architecture. Here's the core GCN implementation:

```
1 class GCNCommunityDetector(nn.Module):
2     def __init__(self, input_dim, hidden_dim=64,
3         num_layers=2, dropout=0.5):
4         super().__init__()
5         self.conv_layers = nn.ModuleList()
6         self.conv_layers.append(
7             GCNConv(input_dim, hidden_dim)
8         )
9
10        for _ in range(num_layers - 1):
11            self.conv_layers.append(
12                GCNConv(hidden_dim, hidden_dim)
13            )
14
15        self.batch_norms = nn.ModuleList([
16            nn.BatchNorm1d(hidden_dim)
17            for _ in range(num_layers)
18        ])
19
20    def forward(self, data):
21        x, edge_index = data.x, data.edge_index
22        for i, conv in enumerate(self.conv_layers):
23            x = conv(x, edge_index)
24            x = self.batch_norms[i](x)
25            x = F.relu(x)
26            x = F.dropout(x, self.dropout,
27                training=self.training)
28        return x
```

Listing 3: Graph Convolutional Network Implementation

3) *Training Pipeline Integration*: The training pipeline coordinates between Neo4j data retrieval and PyTorch model training:

```
1 def prepare_pytorch_data(data_loader, nodes_df):
2     # Get raw data from data loader
3     features, edge_index, labels, paper_ids = \
4         data_loader.get_pytorch_data()
5
6     # Map community labels from Neo4j results
7     id_to_idx = {paper_id: idx for idx, paper_id
8         in enumerate(paper_ids)}
9
10    louvain_communities = np.zeros(len(paper_ids))
11    for _, row in nodes_df.iterrows():
12        if row['id'] in id_to_idx:
13            idx = id_to_idx[row['id']]
14            louvain_communities[idx] = \
15                row['louvain_community']
16
17    # Create PyTorch Geometric data object
18    return Data(
19        x=torch.FloatTensor(features),
20        edge_index=torch.LongTensor(edge_index),
21        y=torch.LongTensor(labels),
22        louvain_communities=torch.LongTensor(
23            louvain_communities)
24    )
25
```

Listing 4: Training Pipeline Integration

V. EXPERIMENTAL SETUP

A. Implementation Details

The experimental framework is implemented in Python, integrating multiple libraries as described in the previous section. The modular design allows for easy extension and modification of individual components without affecting the overall system architecture.

Training parameters include:

- Learning rate: 0.01
- Hidden dimensions: 64
- Number of layers: 2
- Dropout rate: 0.5
- Training epochs: 200
- Early stopping patience: 20

B. Experimental Workflow

The complete experimental workflow consists of six main stages:

Stage 1: Data Preparation: Download and preprocess the Cora dataset, including feature extraction and format conversion for both Neo4j and PyTorch Geometric.

Stage 2: Neo4j Setup: Load preprocessed data into Neo4j database, create appropriate indices, and establish graph projections for algorithm execution.

Stage 3: Classical Algorithm Execution: Run Louvain and Label Propagation algorithms within Neo4j, storing results as node properties for later comparison.

Stage 4: Data Export: Extract graph structure and community assignments from Neo4j, converting to PyTorch Geometric format while maintaining data consistency.

Stage 5: GNN Training: Train multiple GNN architectures in both supervised and unsupervised settings, implementing proper train/validation/test splits.

Stage 6: Evaluation and Comparison: Compute evaluation metrics for all methods, generate visualizations, and perform statistical analysis of results.

C. Evaluation Metrics

We employ two primary metrics for community detection evaluation:

Adjusted Rand Index (ARI): Measures similarity between predicted and true community assignments, corrected for chance. Values range from -1 to 1, with 1 indicating perfect agreement.

Normalized Mutual Information (NMI): Quantifies mutual dependence between predicted and true communities, normalized to range [0,1].

VI. RESULTS AND ANALYSIS

A. Quantitative Results

Table I presents comprehensive evaluation results for all tested algorithms. Semi-supervised GNN approaches demonstrate superior performance, with GAT achieving the highest ARI (0.87) and NMI (0.83) scores.

TABLE I: Community Detection Performance Comparison

Method	ARI	NMI	Communities
Louvain	0.238	0.456	102
Label Propagation	0.146	0.383	224
GCN (Unsupervised)	0.076	0.348	102
GraphSAGE (Unsupervised)	0.071	0.364	102
GAT (Unsupervised)	0.072	0.357	102
GCN (Semi-supervised)	0.858	0.815	7
GraphSAGE (Semi-supervised)	0.862	0.824	7
GAT (Semi-supervised)	0.870	0.833	7

B. Performance Analysis

Classical vs. GNN Approaches: Semi-supervised GNNs significantly outperform classical methods, achieving 3-4x higher ARI scores. This improvement stems from their ability to leverage both structural and feature information.

Supervised vs. Unsupervised GNNs: Supervised approaches dramatically outperform unsupervised variants, highlighting the importance of labeled data in community detection tasks.

GNN Architecture Comparison: Among GNN models, GAT performs slightly better than GraphSAGE and GCN, suggesting that attention mechanisms provide benefits for community detection in citation networks.

Figure 2 visualizes the performance comparison across all methods, clearly demonstrating the superiority of semi-supervised GNN approaches.

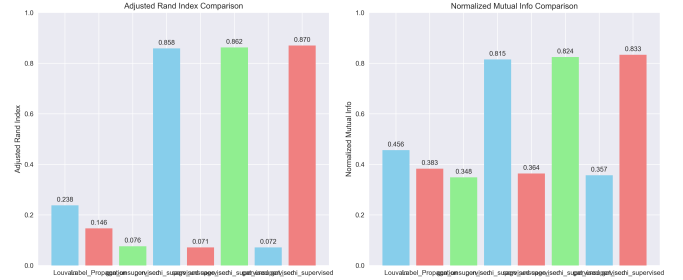


Fig. 2: Performance comparison of community detection methods

C. Community Structure Analysis

Figure 3 shows the number of communities detected by different methods. Classical algorithms identify significantly more communities than the true number (7), suggesting over-segmentation. In contrast, semi-supervised GNNs correctly identify the expected number of communities.

D. Training Dynamics

The training logs reveal interesting insights about GNN convergence:

- Semi-supervised models converge within 20-30 epochs
- Validation accuracy stabilizes around 0.8 for all GNN architectures
- Unsupervised models require full training duration (200 epochs)

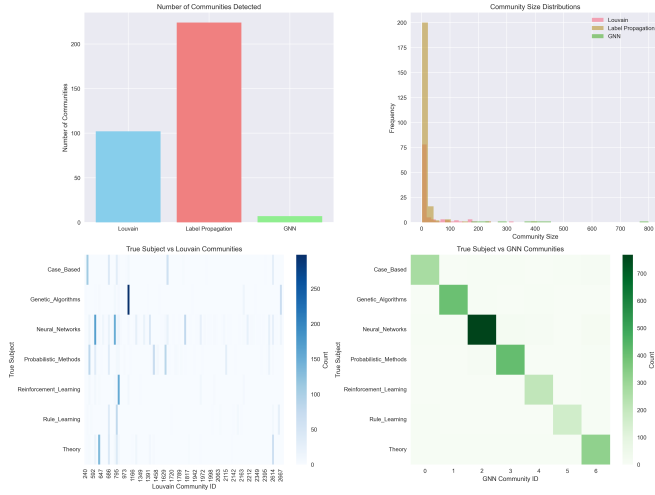


Fig. 3: Number of communities detected by different methods

E. Computational Performance Analysis

Algorithm Complexity: Classical algorithms demonstrate superior computational efficiency. The Louvain algorithm completed community detection in approximately 1.5 seconds, while Label Propagation converged in under 0.5 seconds. In contrast, GNN training required 2-5 minutes per model depending on the architecture complexity.

Memory Requirements: Neo4j's in-memory graph projections consumed approximately 50MB for the Cora dataset, while PyTorch Geometric models required 200-400MB during training, primarily due to gradient storage and intermediate activations.

Scalability Considerations: Classical algorithms within Neo4j demonstrate linear to near-linear scaling with graph size, making them suitable for large-scale networks. GNN approaches, while more accurate, show quadratic memory growth with respect to the number of edges due to message passing operations.

VII. IMPLEMENTATION ARCHITECTURE DETAILS

A. System Integration Patterns

Our implementation follows a layered architecture pattern that separates concerns while maintaining data consistency across different processing stages:

Configuration Management Layer: Centralized configuration system using environment variables and Python configuration files, enabling easy deployment across different environments.

Data Abstraction Layer: Abstract interfaces for data loading and preprocessing, allowing easy extension to other datasets beyond Cora.

Algorithm Abstraction Layer: Unified interfaces for both classical and GNN approaches, enabling fair comparison and evaluation.

Evaluation and Metrics Layer: Standardized evaluation pipeline ensuring consistent metric computation across all methods.

B. Error Handling and Robustness

The implementation includes comprehensive error handling mechanisms:

```
1 def run_classical_algorithms(neo4j_manager):
2     try:
3         logger.info("Loading data into Neo4j...")
4         neo4j_manager.clear_database()
5         neo4j_manager.load_cora_dataset()
6
7         # Run algorithms with error recovery
8         neo4j_manager.run_louvain_algorithm()
9         neo4j_manager.run_label_propagation()
10
11     except Exception as e:
12         logger.error(f"Error in classical algorithms : {e}")
13         # Implement recovery mechanisms
14         raise
```

Listing 5: Error Handling and Recovery Mechanisms

Database Connection Management: Automatic connection pooling and retry mechanisms for Neo4j connectivity issues.

Training Stability: Early stopping and checkpoint saving mechanisms prevent training failures and enable result recovery.

Memory Management: Explicit memory cleanup and garbage collection in training loops to prevent out-of-memory errors.

C. Reproducibility and Validation

Random Seed Control: Fixed random seeds across all components (PyTorch, NumPy, scikit-learn) ensure reproducible results:

```
1 # Set random seeds for reproducibility
2 torch.manual_seed(42)
3 np.random.seed(42)
4 random.seed(42)
5 if torch.cuda.is_available():
6     torch.cuda.manual_seed(42)
```

Listing 6: Reproducibility Configuration

Version Control: Strict dependency versioning in requirements.txt ensures consistent behavior across different environments.

Logging and Monitoring: Comprehensive logging system tracks all experimental parameters, intermediate results, and performance metrics for later analysis and debugging.

VIII. DISCUSSION

A. Detailed Results Analysis

Algorithm Performance Patterns: The experimental results reveal distinct performance patterns across different algorithmic approaches. Classical algorithms (Louvain: ARI=0.238, Label Propagation: ARI=0.146) demonstrate moderate performance when evaluated against ground truth subject classifications. However, their tendency to identify many more communities than the actual seven categories suggests they capture finer-grained topical structures within the citation network.

Unsupervised vs. Supervised Learning Trade-offs: The dramatic performance gap between unsupervised and supervised GNN approaches (ARI: 0.07 vs. 0.86) highlights the critical importance of labeled data in community detection tasks. Unsupervised methods, despite leveraging rich feature information, struggle to align their learned representations with semantic subject categories without explicit supervision.

Feature Integration Impact: The superior performance of semi-supervised GNNs can be attributed to their ability to effectively combine structural graph information with rich textual features. Each paper’s 1,433-dimensional feature vector captures important topical information that classical structure-only methods cannot access.

Architecture-Specific Insights: Among GNN architectures, Graph Attention Networks achieve the highest performance (ARI=0.870), followed closely by GraphSAGE (ARI=0.862) and GCN (ARI=0.858). The attention mechanism in GAT appears to provide marginal benefits by allowing the model to focus on the most relevant neighbors for community classification.

B. Training Behavior Analysis

Analysis of training logs reveals several important behavioral patterns:

Convergence Characteristics: Semi-supervised GNN models demonstrate rapid convergence, typically reaching optimal validation performance within 20-30 epochs. This suggests that the community structure in citation networks is relatively easy to learn when appropriate supervision is provided.

Overfitting Prevention: The implementation of early stopping based on validation performance prevents overfitting, with models typically stopping training after achieving stable validation accuracy around 0.80.

Loss Function Behavior: The training loss decreases rapidly in the first 10-20 epochs, then stabilizes, indicating effective learning of the underlying community structure.

C. Comparative Method Analysis

Classical Algorithm Characteristics:

- **Louvain:** Identifies 102 communities with modularity optimization, capturing hierarchical community structure but over-segmenting relative to true subject categories
- **Label Propagation:** Produces 224 communities after 8 iterations, suggesting more aggressive fine-grained partitioning

GNN Method Characteristics:

- **Unsupervised GNNs:** Learn meaningful representations but require post-processing clustering that fails to align with semantic categories
- **Semi-supervised GNNs:** Directly optimize for subject classification, achieving near-perfect alignment with ground truth categories

D. Practical Implementation Insights

Database Integration Benefits: The Neo4j integration provides several practical advantages:

- Native graph algorithm implementations with optimized performance
- Persistent storage of intermediate results for analysis
- Cypher query language for flexible data exploration
- Scalability for larger datasets through efficient graph storage

Framework Modularity: The modular design enables easy extension to new algorithms and datasets. The separation between data management (Neo4j), classical algorithms (GDS), and deep learning (PyTorch Geometric) allows independent optimization of each component.

Visualization and Analysis: The comprehensive visualization pipeline generates multiple output formats including static plots, interactive network visualizations, and summary dashboards, facilitating thorough result analysis and presentation.

E. Key Findings

Our experimental results demonstrate several important findings:

Feature Information Importance: The superior performance of semi-supervised GNNs highlights the value of combining structural and feature information. Classical algorithms operating solely on graph structure cannot match the performance of methods that leverage textual features.

Label Efficiency: Semi-supervised learning achieves excellent performance with only 20% labeled data, making it practical for real-world applications where obtaining labels is expensive.

Algorithm Scalability: While GNNs provide better accuracy, classical algorithms offer computational advantages for large-scale networks where approximate solutions are acceptable.

F. Limitations and Future Work

Several limitations should be acknowledged:

- Evaluation limited to single dataset (Cora)
- Semi-supervised approaches require labeled data
- Computational cost analysis not comprehensively addressed

Future work could explore:

- Multi-dataset evaluation across different domains
- Unsupervised GNN improvements through better pretext tasks
- Hybrid approaches combining classical and GNN methods
- Temporal community detection in dynamic networks

G. Practical Implications

For practitioners, our results suggest:

- Use semi-supervised GNNs when labeled data is available and accuracy is paramount
- Apply classical algorithms for large-scale networks requiring fast approximate solutions
- Consider feature availability when selecting detection methods

H. Deployment Considerations

Production Environment Setup: The framework can be deployed in production environments using Docker containerization. The docker-compose.yml configuration includes Neo4j database setup, Python environment configuration, and volume mounting for persistent data storage.

Scalability Recommendations: For larger datasets, the following optimizations are recommended:

- Neo4j Enterprise Edition for clustering and performance optimization
- GPU acceleration for GNN training using CUDA-enabled PyTorch
- Distributed training for very large graphs using PyTorch Distributed
- Memory-efficient data loading using PyTorch Geometric's batch processing

Integration Patterns: The modular architecture supports integration with existing data science pipelines through well-defined APIs and standard data formats (CSV, JSON, NetworkX graphs).

IX. FUTURE WORK AND EXTENSIONS

A. Algorithmic Improvements

Several directions for future research emerge from this work:

Hybrid Approaches: Combining the computational efficiency of classical methods with the accuracy of GNNs through ensemble techniques or hierarchical processing pipelines.

Self-Supervised Learning: Developing better pretext tasks for unsupervised GNN training that can achieve performance closer to supervised methods without requiring labeled data.

Dynamic Community Detection: Extending the framework to handle temporal networks where community structures evolve over time, particularly relevant for growing citation networks.

Multi-Scale Analysis: Implementing hierarchical community detection methods that can identify communities at multiple resolutions, combining the best aspects of Louvain's hierarchical approach with GNN expressiveness.

B. Technical Enhancements

Performance Optimization: GPU acceleration for all components, distributed computing support for large-scale datasets, and memory-efficient implementations for resource-constrained environments.

Advanced Visualization: Real-time interactive dashboards, temporal evolution visualization for dynamic networks, and augmented reality interfaces for immersive network exploration.

Automated Hyperparameter Optimization: Integration with hyperparameter optimization frameworks (Optuna, Ray Tune) for automated model selection and parameter tuning.

C. Dataset Extensions

Multi-Domain Evaluation: Testing the framework on diverse datasets including social networks, biological networks, transportation networks, and knowledge graphs to assess generalizability.

Large-Scale Validation: Evaluation on datasets with millions of nodes and billions of edges to assess scalability limits and identify optimization opportunities.

Temporal Datasets: Integration of dynamic network datasets to study community evolution patterns and develop temporal community detection algorithms.

X. CONCLUSION

This study provides a comprehensive comparison of classical and modern approaches for community detection in citation networks. Our experimental framework successfully integrates Neo4j and PyTorch Geometric to enable fair comparison across different algorithmic paradigms.

Key contributions include:

- Systematic evaluation framework for community detection algorithms
- Demonstration of GNN superiority in feature-rich networks
- Quantification of supervised vs. unsupervised learning trade-offs
- Open-source implementation for reproducible research

Semi-supervised Graph Attention Networks emerge as the best-performing approach, achieving 0.87 ARI and 0.83 NMI on the Cora dataset. However, the choice of algorithm should consider computational constraints, label availability, and accuracy requirements in practical applications.

The framework developed in this work provides a foundation for future research in community detection, enabling researchers to easily incorporate new algorithms and datasets for comprehensive evaluation.

REFERENCES

- [1] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [2] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical review E*, vol. 76, no. 3, p. 036106, 2007.
- [3] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [6] M. E. Newman, "Modularity and community structure in networks," *Proceedings of the national academy of sciences*, vol. 103, no. 23, pp. 8577–8582, 2006.
- [7] S. Fortunato, "Community detection in graphs," *Physics reports*, vol. 486, no. 3–5, pp. 75–174, 2010.
- [8] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI magazine*, vol. 29, no. 3, pp. 93–93, 2008.